



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №3
Технології розробки програмного забезпечення
«ДІАГРАМА РОЗГОРТАННЯ. ДІАГРАМА КОМПОНЕНТІВ.
ДІАГРАМА ВЗАЄМОДІЙ ТА ПОСЛІДОВНОСТЕЙ»

Виконав:
студент групи ІА-24
Іскандаров Е. Е.
Перевірив:
Мягкий М. Ю.

Київ 2024

Зміст

Короткі теоретичні відомості	3
Хід роботи.....	5
Діаграма розгортання для проектованої системи	5
Діаграма компонентів для розроблюваної системи	7
Діаграма послідовностей для проектованої системи	8
Посилання на репозиторій	10
Висновок.....	10

Тема: ДІАГРАМА РОЗГОРТАННЯ. ДІАГРАМА КОМПОНЕНТІВ. ДІАГРАМА ВЗАЄМОДІЙ ТА ПОСЛІДОВНОСТЕЙ.

Мета: Розробити діаграми розгортання, компонентів, взаємодій та послідовностей для обраної теми проекту

Короткі теоретичні відомості

Діаграма розгортання (Deployment Diagram)

Мета:

Діаграма розгортання в UML використовується для моделювання фізичного розташування програмного забезпечення і апаратного забезпечення. Вона показує, як компоненти системи розгортаються на вузлах (сервери, клієнти тощо).

Ключові елементи:

- **Вузли (Nodes):** Фізичні пристрої або віртуальні сервери, де розгортається програмне забезпечення.
- **Артефакти (Artifacts):** Програмні модулі (наприклад, файли, бази даних), які розгортаються на вузлах.
- **Зв'язки (Communication Paths):** Лінії між вузлами, що показують обмін даними.

Приклад застосування:

- Розподіл серверних компонентів між веб-сервером, базою даних і клієнтськими пристроями.

Діаграма компонентів (Component Diagram)

Мета:

Діаграма компонентів використовується для представлення структури системи з точки зору її модулів або компонентів. Вона показує, як ці компоненти взаємодіють один з одним і з зовнішнім середовищем.

Ключові елементи:

- **Компоненти (Components):** Логічні модулі, що виконують певні функції.
- **Інтерфейси (Interfaces):** Визначають точки взаємодії компонентів.
- **Залежності (Dependencies):** Показують, як один компонент залежить від іншого.

Приклад застосування:

- Визначення зв'язків між мікросервісами у розподіленій архітектурі.

Діаграма взаємодій (Interaction Diagram)

Мета:

Діаграма взаємодій відображає, як об'єкти системи взаємодіють між собою для виконання певного сценарію.

Типи:

1. Діаграма послідовностей (Sequence Diagram):

- **Мета:** Показує порядок взаємодії об'єктів у часі.
- **Ключові елементи:**
 - **Актори (Actors):** Ініціюють взаємодії.
 - **Об'єкти (Objects):** Елементи системи, які беруть участь у процесі.
 - **Повідомлення (Messages):** Вказують передачу даних між об'єктами.
- **Приклад:** Авторизація користувача в системі.

2. Діаграма комунікацій (Communication Diagram):

- **Мета:** Фокусується на зв'язках між об'єктами.
- **Ключові елементи:**
 - **Вузли (Nodes):** Об'єкти, які взаємодіють.
 - **Повідомлення:** Лінії, які показують передачу даних між вузлами.

Приклад застосування:

- **Діаграма послідовностей:** Моделювання процесу оформлення замовлення.
- **Діаграма комунікацій:** Представлення зв'язків між сервісами в мікросервісній архітектурі.

Ці діаграми в поєднанні допомагають зрозуміти архітектуру системи, її компоненти, фізичну інфраструктуру та взаємодію між різними елементами в часі.

Хід роботи

Web-browser (proxy, chain of responsibility, factory method, template method, visitor, p2p) Веб-браузер повинен мати можливість зробити наступне: мати адресний рядок для введення адреси сайту, переміщатися і відображати структуру html документа, переглядати підключений javascript та css файли, перегляд всіх підключених ресурсів (зображень), коректна обробка відповідей з сервера (коди відповідей HTTP) - переходи при перенаправленнях, відображення сторінок 404 і 502/503.

Діаграма розгортання для проектованої системи

На діаграмі розгортання зображаємо фізичні компоненти, необхідні для роботи системи. Це сервер, на якому буде встановлено Backend та клієнтська частина.

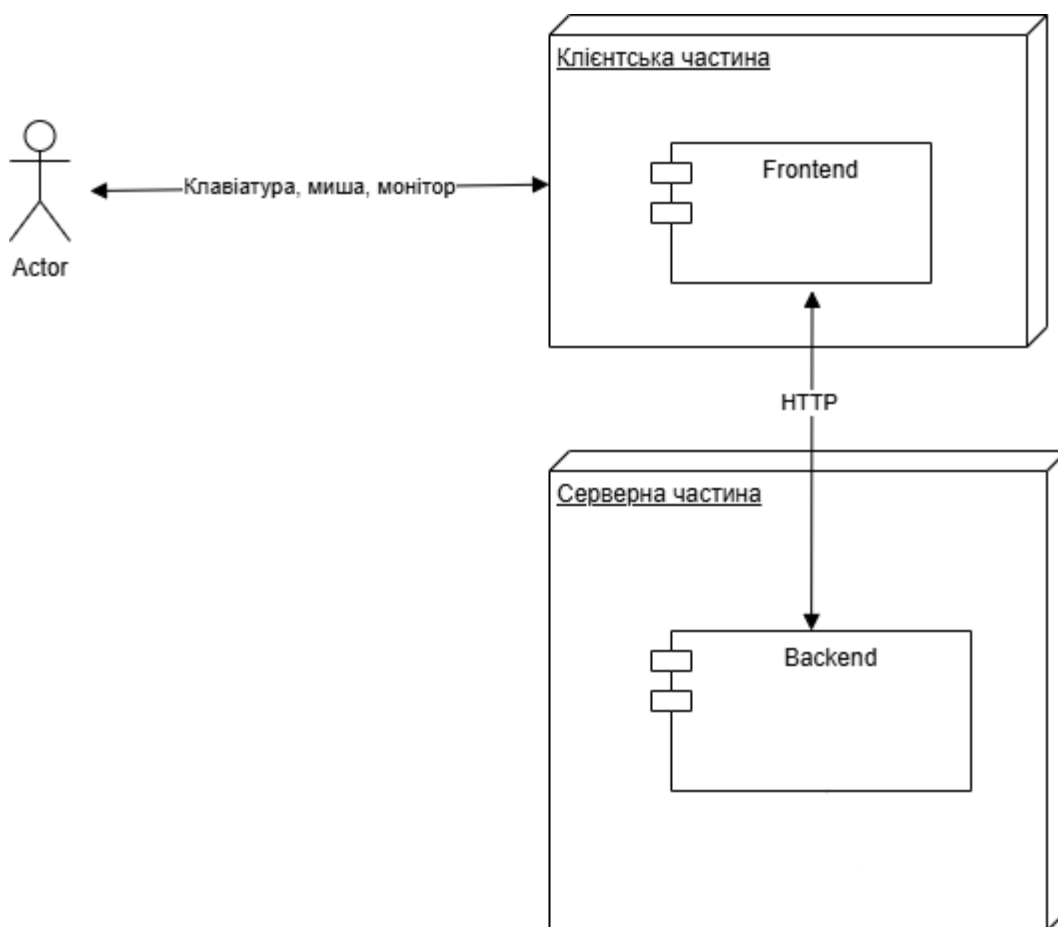


Рисунок 1 — Діаграма розгортання

Опис компонентів:

1. Actor (Користувач):

- Представляє зовнішнього користувача системи.
- Використовує клієнтську частину системи через клавіатуру, мишу і монітор.

2. Клієнтська частина:

- Відповідає за взаємодію з користувачем.
- Надсилає запити до серверної частини через протокол HTTP.
- Містить інтерфейс для введення і відображення даних.

3. Серверна частина:

- Основний компонент для обробки логіки додатка.
- Приймає HTTP-запити від клієнтської частини, обробляє їх і надсилає відповіді.
- Використовує JDBC (Java Database Connectivity) для взаємодії з базою даних.

Взаємодії:

- **Actor -> Клієнтська частина:** Користувач взаємодіє з клієнтською частиною через пристрої введення (клавіатура, миша) та пристрої відображення (монітор).
- **Клієнтська частина -> Серверна частина:** Клієнтська частина надсилає HTTP-запити до серверної частини для виконання бізнес-логіки та отримання даних.

Основні протоколи та інструменти:

- **HTTP:** Використовується для передачі даних між клієнтською і серверною частиною.

Ця архітектура ілюструє типову структуру клієнт-серверного додатка із базою даних. Якщо потрібна додаткова інформація або уточнення, повідомте!

Діаграма компонентів для розроблюваної системи

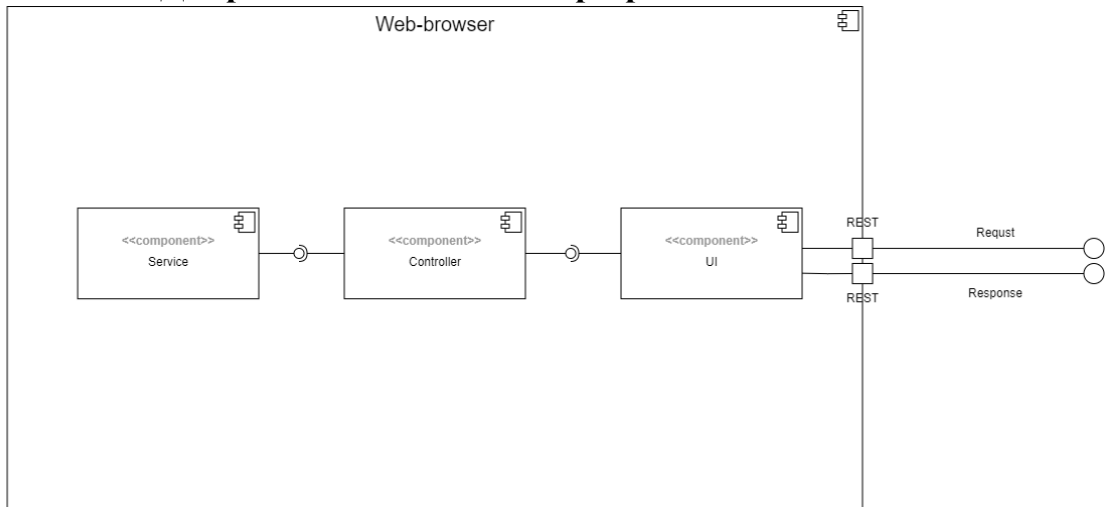


Рисунок 2 — Діаграма компонентів

UML-діаграма компонентів, яка ілюструє архітектуру системи.

- **UI Components** (Компоненти інтерфейсу): Представляють елементи інтерфейсу, які забезпечують взаємодію з користувачем та відображають зворотний зв'язок.
- **Controller** (Контролер): Слугує проміжним шаром, який обробляє REST-запити та передає їх відповідному сервісу.
- **Service** (Сервіс): Представляє бізнес-логіку застосунку.

Компоненти пов'язані через REST-запити та відповіді, а взаємодія користувача відображена у вигляді циклів вводу та зворотного зв'язку.

Діаграма послідовностей для проектованої системи

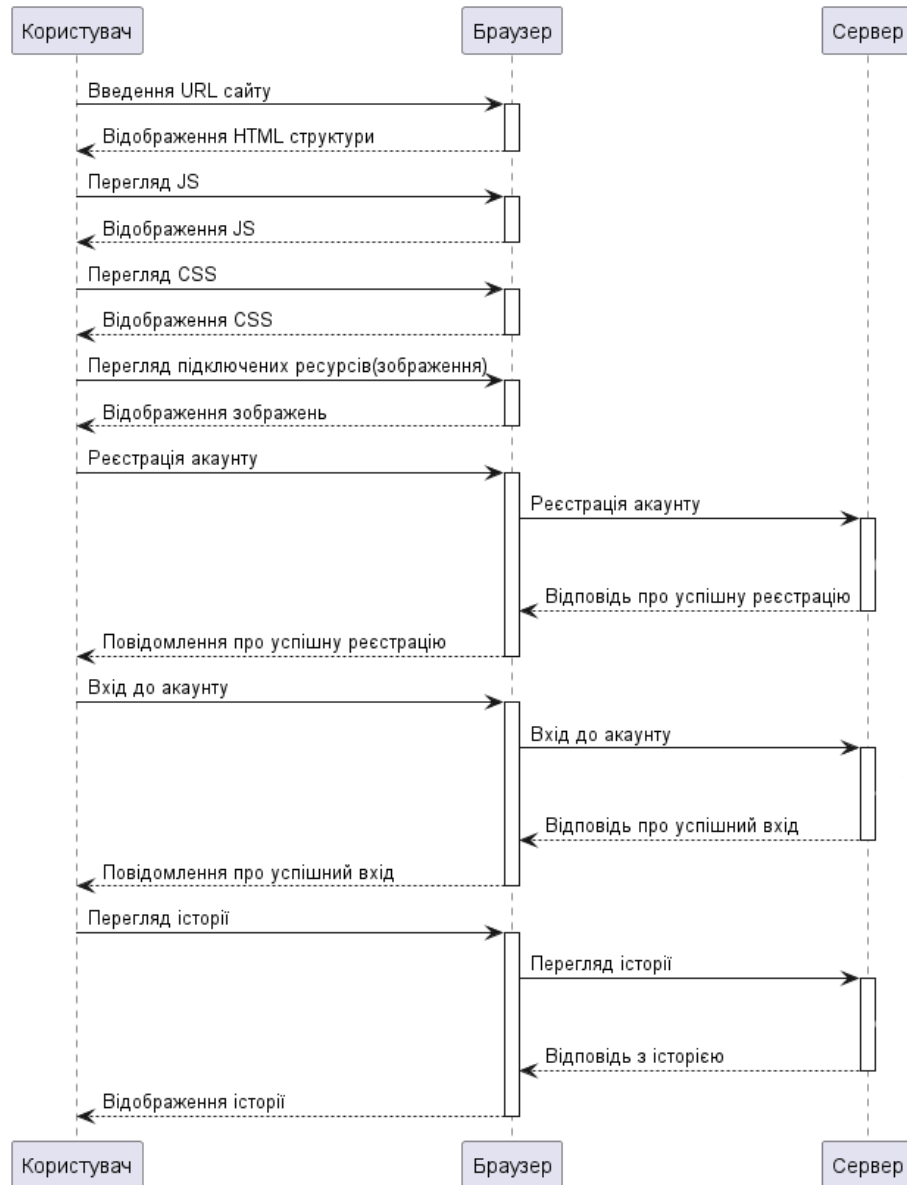


Рисунок 3 — Діаграма послідовностей

Опис діаграми:

1. Клієнтська частина:

- Користувач вводить URL сайту для завантаження веб-сторінки.
- Відображається HTML-структура сторінки у браузері.
- Користувач переглядає JavaScript-елементи, які обробляються браузером.
- Відображається результат роботи JavaScript.
- Переглядаються CSS-файли, які застосовуються до структури сторінки.
- Відображаються стилі, що визначені у CSS.
- Користувач переглядає підключені ресурси, наприклад, зображення.

- Після завантаження зображення відображаються у браузері.
- Для реєстрації користувач вводить дані, які передаються на сервер.

2. Браузер:

- Передає дані реєстрації на сервер.
- Після отримання підтвердження про успішну реєстрацію відображає повідомлення для користувача.
- Для входу в акаунт дані авторизації передаються на сервер.
- Після отримання підтвердження успішного входу відображає повідомлення про це користувачу.
- Надсилає запит на сервер для перегляду історії користувача.
- Відображає історію після отримання відповіді від сервера.

3. Серверна частина:

- Отримує дані для реєстрації акаунту від браузера.
- Формує та зберігає новий запис у базі даних.
- Надсилає підтвердження успішної реєстрації до браузера.
- Отримує дані для входу в акаунт і перевіряє їх у базі даних.
- Надсилає відповідь про успішний вхід у разі коректності даних.
- Приймає запит на перегляд історії та отримує відповідну інформацію з бази даних.
- Повертає відповідь із історією до браузера.

Альтернативний сценарій:

- У випадку некоректних даних для реєстрації чи входу сервер надсилає повідомлення про помилку, яке відображається у браузері.

Посилання на репозиторій:

https://github.com/Elmir2/trpz_labs

Висновок: У даній лабораторній роботі було створено діаграми послідовностей, розгортання та компонентів. Це дозволяє краще зрозуміти архітектуру системи, процеси її роботи та взаємодію між складовими.

Діаграма послідовностей деталізує взаємодії між користувачем, клієнтською частиною, сервером, визначаючи основні етапи авторизації, перевірки прав доступу та створення даних у системі. Особливу увагу приділено сценаріям обробки помилок.

Діаграма розгортання відображає фізичне розташування компонентів системи, взаємодію між клієнтською та серверною частинами. Це дає змогу зрозуміти, як система функціонує у реальному середовищі.

Діаграма компонентів описує логічну структуру системи, включаючи модулі, сервіси, контролери та взаємодію з базою даних. Вона допомагає зрозуміти, як програмні частини об'єднуються в єдину архітектуру, і показує функції кожного модуля.

Створені діаграми є основою для документування архітектури системи, полегшують комунікацію між розробниками та зацікавленими сторонами, а також сприяють побудові технічного завдання та оптимізації процесів